

Density based learned spatial index for clustered data

Xiaofei Zhao^{*,} Kam-Yiu Lam

Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong

ARTICLE INFO

MSC:

68P20

68P10

68P05

Keywords:

Learned index

Spatial index

Clustered data

Density-based partitioning

ABSTRACT

Retrieving spatial points, such as GPS records or Point of Interests, that satisfy specific location-based query criteria is a core operation in location-based services. Recent studies have shown that learned indexes can outperform traditional indexing methods in both query performance and space efficiency by leveraging data distribution to construct compact predictive models. On the other hand, traditional indexes typically make minimal assumptions about the underlying data distribution. In real-world spatial databases, data is often non-uniformly distributed and tends to cluster in specific regions or along road networks. Adaptivity to such data patterns may bring performance benefits.

In this paper, we explore the construction of efficient learned indexes that exploit the clustering characteristics of spatial datasets. Specifically, we propose a Density-based Grid Learning Spatial Index (DGLSI), which partitions the spatial domain based on point density and utilizes learned models, including multiple recursive model indexes to predict the grid cell IDs of query points. We evaluate DGLSI's performance on real-world GPS datasets and demonstrate that the proposed methods outperform analogous grid-based indexes across various query workloads, including nearest point queries and range queries while maintaining high space efficiency.

1. Introduction

Spatial data indexing, which enables the efficient retrieval of spatial points from large-scale spatial databases, is a fundamental component of location-based services (LBS) and geographic information systems (GIS). For instance, in an online digital map service, users frequently issue queries to locate nearby points of interest (POIs), such as restaurants, gas stations, or hospitals, based on their current location. Each POI can be represented as a spatial point enriched with attributes such as name, address, and operating hours. Given the high density of both POIs and users in urban environments, it is essential to design spatial indexing structures that support efficient query processing. Here, “efficient” refers to both low query response time, which ensures a satisfactory user experience, and low query cost, which helps minimize system load.

Unlike traditional data, spatial data is inherently multidimensional (e.g., 2D or 3D coordinates), posing unique challenges for indexing and query processing. Although various classical techniques exist for indexing one-dimensional (linear) data, they are generally not well-suited for multidimensional data and require substantial modification to be effective. We propose our spatial index method called *Density-based Grid Learning Spatial Index* (DGLSI), which combines the strengths of traditional spatial indexing methods and learned indexes while addressing the locality issues in space-filling curves (SFCs) and the challenges of maintaining the index when the dataset is updated.

The remainder of this paper is organized as follows. Section 2 provides the definitions and the background concepts. Section 3 details the proposed DGLSI index structure, data layout, index building process and query processing algorithms. Section 5 presents an evaluation that analyzes the storage overhead, query performance, and other trade-offs of DGLSI compared to prior methods.

2. Preliminaries

In this section, we begin by reviewing traditional spatial indexing methods. Motivated by the introduction of learned indexes by Kraska et al. [1], we then explore their extension to the spatial domain. To support efficient data updates without retraining the model on the entire dataset, we adopt the concept of out-of-place updates, inspired by the design of Log-Structured Merge Trees (LSM-trees).

2.1. Traditional spatial indexing approaches

The two mainstream classic families of spatial indexing approaches are the R-tree family [2,3] and space-filling curve (SFC) based indexes [4].

The R-tree family represents a direct approach to indexing multidimensional data without transforming it into a linear representation. It

* Corresponding author.

E-mail addresses: xf.zhao@my.cityu.edu.hk (X. Zhao), cskylam@cityu.edu.hk (K.-Y. Lam).

employs a hierarchical tree structure in which each node corresponds to a bounding rectangle. Specifically, each leaf node corresponds to the *minimum bounding rectangle* (MBR) that encloses a relatively small set of spatial objects in the database, while each non-leaf node stores the MBR that covers all its child nodes. This hierarchical organization facilitates efficient spatial queries. For spatial range queries, i.e., retrieving all spatial objects within a specified rectangular region (the “query rectangle”), the R-tree is accessed in a top-down manner, starting from the root. At each level, the algorithm checks which child nodes intersect the query rectangle and recursively descends into those subtrees. This process continues until the leaf nodes are reached, from which all spatial objects satisfying the query conditions are returned. Note that multiple leaf nodes may need to be accessed, as the query rectangle can intersect several MBRs.

On the other hand, space-filling curve (SFC) based indexes take a different approach by transforming the multidimensional spatial data into a one-dimensional representation. A space-filling curve is a continuous one-dimensional curve that, in theory, passes through every point in a multidimensional space. In practice, the service area is divided into a grid of finitely many cells, and an SFC is chosen to traverse all cells in a predefined order. Due to the one-dimensional nature of the curve, each cell can be assigned a unique identifier based on its position along the curve, allowing the cells to be sorted linearly. This transformation reduces the spatial indexing problem to a one-dimensional indexing problem, for which many well-established techniques, such as B-trees and hash indexes, can be effectively applied.

Some significant examples of the above two approaches are summarized in Section 6.

Both approaches have distinct strengths and limitations. (1) R-trees are well-suited to adapting to data distributions through carefully designed node-splitting algorithms. However, the size of an R-tree index can grow substantially with the number of data entries, resulting in significant storage overhead. (2) In contrast, SFC-based indexes offer high space efficiency, since the indexing mechanism functions more as a sorting scheme for grid cells rather than as a standalone data structure. Nevertheless, SFC-based methods suffer from locality issues: spatially proximate points in the original 2D space may be mapped to distant positions in the 1D space. This loss of locality can significantly degrade query performance, particularly for range queries, due to an increased number of unnecessary disk accesses.

2.2. Learned indexes for spatial data

In 2018, Kraska et al. [1] introduced the concept of learned indexes. The core idea is to treat an index as a model that takes a query key as input and outputs the “predicted” physical position of the corresponding data record. Unlike traditional index structures such as B-trees, learned indexes employ machine learning models to perform this mapping. These models have demonstrated promising results, significantly reducing storage overhead while maintaining, or even improving query performance, thereby offering a viable alternative to conventional indexing techniques like B-trees and hash indexes. Although the original learned index framework [1] was designed for one-dimensional data, extending this approach to multidimensional indexing is a natural and increasingly active area of research.

Intuitively, multidimensional data such as spatial points can be transformed into a one-dimensional space using a space-filling curve (SFC), such as the Z-curve, prior to applying a one-dimensional learned index. Several studies have explored this idea, including the ZM index [5], SPRIG [6], and LPBF [7].

However, the *locality issues* inherent in SFC-based indexes [8] remain unresolved and may be further exacerbated in the context of learned indexes. The locality problem arises when two spatial points, p_1 and p_2 , that are close in the original 2D space are mapped to distant positions in the transformed 1D space. As illustrated in Fig. 1, consider a range query $Q(p_1, p_2)$ that retrieves all points within the rectangle

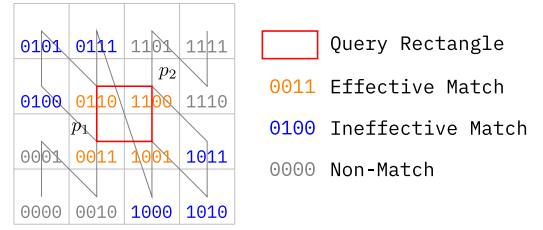


Fig. 1. Locality problem in SFC-based indices.

defined by p_1 (bottom left) and p_2 (top right). While the physical storage locations of the corresponding cells for p_1 and p_2 (e.g., 0011 and 1100) can be efficiently determined in the sorted 1D dataset, other relevant cells, such as 0110 and 1001, may also contain matching records but are not stored near p_1 or p_2 in 1D space. Learned indexes use predictive models to estimate the storage location of a record. When the prediction is inaccurate, they rely on a fallback mechanism known as *last-mile search*, which extends the search to neighboring positions. If “neighborhood” is defined solely by cell IDs in the 1D space, the lack of spatial locality can result in many unnecessary cell accesses, as illustrated by the ineffective matches in Fig. 1.

2.3. Out-of-place updates

Another major challenge in learned indexes is the overhead associated with maintaining the index under data updates. Training a learned model is typically a time-consuming, one-time process. However, in real-world storage systems, the index must be updated as new data is inserted or existing data is deleted. Retraining the model from scratch for every update is impractical, making it essential to design mechanisms that support efficient incremental updates. We may benefit from the concept of *out-of-place updates*, borrowed from modern storage systems such as log-structured merge trees (LSM trees) [9].

The out-of-place update mechanism is a widely adopted strategy in modern write-intensive storage systems. For example, in a simple key-value store, the data records are normally sorted by their keys, and the majority of the data records are serialized in a file on the persistent storage (e.g., SSD or HDD; except some cases where in-memory database is used). However, the insertion of new data entries may not follow the sorted order, which means expensive random writes are required to insert the new data while maintaining the sorted order. To avoid this, the key idea of out-of-place updates is to buffer the new data entries in a memory buffer, where random access and data sorting are much faster than on the persistent storage. Once the buffer reaches a certain size, the buffered data is flushed to the persistent storage in a sequential manner, which is faster than random writes.

2.4. Clustered data distribution

Moreover, there has been limited discussion in the literature on how the performance of learned indexes is influenced by data distribution and model parameter choices. A deeper understanding of these factors is critical for ensuring robust and adaptive indexing in dynamic environments.

In real-world LBS and spatial database applications, spatial data points are typically distributed non-uniformly. Fig. 2(a) shows a set of randomly generated, uniformly distributed spatial points, whereas Fig. 2(b) presents a real-world example of road network nodes in San Francisco, extracted from OpenStreetMap. In many practical scenarios, such as points of interest (PoIs) or vehicle GPS trajectories, data points exhibit significant clustering, often along roads or in urban centers. In such cases, the effectiveness of uniform grid-based indexing may degrade.

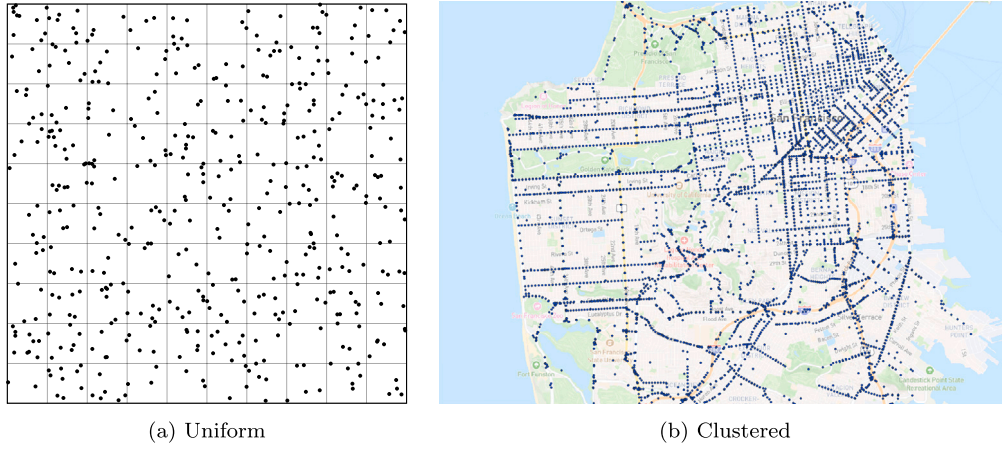


Fig. 2. Different data distributions.

To simplify the dataset, some studies apply map-matching techniques to align spatial points to the road network. However, such strategies may not be suitable for applications where preserving the exact location of points is critical. In this study, we do not employ map-matching; instead, we work directly with raw spatial data.

3. DGLSI

With the above considerations in Section 2 in mind, in this study, we propose a new learned spatial index that has the following properties:

1. *Simple storage structure*: The index is based on a grid partitioning of the service area, similar to SFC-based methods.
2. *Adaptive*: The grid partitioning is guided by the data distribution rather than being uniformly spaced. This enables the index to better accommodate clustered data patterns, which are prevalent in real-world scenarios such as GPS trajectory data.
3. *Learned*: The index is constructed using learned models. Traditional SFC-based methods are not directly applicable to non-uniform grids, necessitating an alternative indexing mechanism. We adopt learned models to minimize storage overhead while preserving efficient access.
4. *Updatable*: The index supports efficient updates, allowing new data to be incorporated without retraining the model from scratch.

Table 1 summarizes the notations that are frequently used in this paper.

Definition 1 (Spatial Data Entry and Spatial Key). A spatial data entry is an encoded string consisting of a spatial key $\mathbf{k}$ and an associated payload. The spatial key $\mathbf{k}$ is defined by its coordinates $\mathbf{k} = (x, y) \in \mathbb{R}^2$, where $L_0 \leq x \leq U_0$ and $L_1 \leq y \leq U_1$, i.e., within the bounds of the service area.

For example, to store GPS points using longitude and latitude as the spatial representation, we can set $L_0 = -180$, $U_0 = 180$, $L_1 = -90$, and $U_1 = 90$.

Note that we assume the payload is also a fixed-size string, which is a reasonable assumption in many scenarios, for example, when storing GPS points in a predefined format with fixed attributes.

3.1. Objective: Spatial queries

The proposed method aims to support the following two types of spatial queries:

Table 1
Notations.

Notation	Description
p_i	A spatial point (2-D unless stated otherwise) with ID i .
$\mathbf{V}$	$\mathbf{V} = [L_0, U_0] \times [L_1, U_1] \subseteq \mathbb{R}^2$, the service area, a subspace of $\mathbb{R}^2$.
$\mathbf{k}$	The key for a data entry. In the context of index, we focus on the key and the position of a data entry, so we may use the terminologies k , data entry/record/point interchangeably.
location	We use the term to refer to the spatial location of a data point.
$pos, pos_{pred}, pos_{real}$	Different from location, we use “position” to refer to the physical storage position (predicted or real) of a data record, e.g., the byte offset in a file or an array.
$Q_c, Q(p_c, r)$	A spatial query that asks for the spatial points within a circle centered at point p_c and with radius r .
N	The total number of data entries in the dataset.
N_c	The threshold for the number of data entries in each column and row of the grid.
n	The (expected) number of data entries in a grid cell.
$M(\mathbf{k})$	A learned model that takes $\mathbf{k}$ as its input and predicts the position of the corresponding data record. May give inaccurate predictions.

1. *Nearest neighbor query* $Q(p_c, r)$: Given a query point p_c and an optional radius parameter r , this query returns the spatial point p in the database that is nearest to p_c , subject to the constraint $d(p, p_c) \leq r$, where $d()$ denotes the distance function. This query type naturally extends to the k -nearest neighbor (k-NN) query. When the radius r is omitted, the query is denoted as Q_c .
2. *Range query* $Q(p_1, p_2)$: Given two points p_1 and p_2 that define a rectangular region in 2D space, this query retrieves all spatial points contained within the rectangle. The result is a set of spatial points satisfying this condition.

3.2. Overview of DGLSI

In this section, we present the key design principles of the DGLSI index structure. We begin with the fundamental concept of grid partitioning, which serves as the foundation of the index. Following an application-driven design approach, we then detail the specific components of DGLSI and discuss how it supports spatial queries.

3.2.1. Density based grid partitioning and numbering

We partition the spatial data into variably-sized grid cells based on data density rather than using a uniform grid. This approach aims to balance the number of data entries in each cell, which is crucial for the

efficiency of the last-mile search in learned indices. Also, we choose the grid-based partitioning approach because it is simple to decide the neighbors of a cell and the spatial range of a cell. Grid cells are kept small, often containing only a few data entries, allowing for efficient retrieval via linear scan.

A grid cell is the basic unit of the DGLSI index structure. We serialize the grid cells into database files and the primary task of DGLSI is to effectively and efficiently *locate the grid cell* that contains the spatial point requested in a spatial query.

As illustrated in Fig. 3, it is important to note that the spacing between vertical and horizontal grid lines is not required to be uniform, i.e., the grid need not follow a Cartesian layout. The only constraint is that all lines must be parallel to the x and y axes, resulting in a *rectilinear* grid. This design choice is motivated by the considerations discussed earlier, for the following reasons: (1) When the cell sizes are fixed, indexing the grid using a space-filling curve is straightforward, and the cell ID can be computed in constant time, i.e., with time complexity $\mathcal{O}(1)$. However, this regular layout lacks adaptability to skewed or clustered data distributions and varying query workloads, which can lead to performance degradation due to data imbalance — some cells may become densely populated while others remain sparse. (2) On the other hand, if the grid is non-rectilinear (i.e., composed of arbitrarily shaped cells), performing last-mile search becomes difficult, as it is non-trivial to determine the spatial range and neighboring cells of a given cell.

Building on the initial concept of the grid, we now consider the query operations that the grid-based index structure support. As previously discussed, the primary objective of the grid design is to enable the following operation:

Definition 2 (Goal of DGLSI). Given a spatial coordinate, efficiently determine the physical storage position of the corresponding grid cell within a database file. The storage position may be a byte offset in the file.

Since the grid cells are serialized into files, they must be assigned linear identifiers and sorted accordingly. Based on this, the overarching “Goal of DGLSI” can be decomposed into two sub-tasks:

- (1) Map the spatial coordinate to its corresponding grid cell ID. This requires a numbering scheme for the grid cells.
- (2) Given the cell ID, determine the physical storage position of the associated grid cell. This is a non-trivial task because cell sizes may vary, making it impossible to compute the byte offset directly from the cell ID. However, since this reduces to a one-dimensional search problem, it can be effectively addressed using a traditional lightweight learned index.

Grid cell numbering. In the first sub-task, a key design consideration for the cell numbering scheme is whether to support cell splitting, as the index structure is intended to accommodate data insertions. When a cell accumulates too many new records, linear scanning within the cell becomes inefficient. In such cases, splitting the cell may be needed. However, in the case where a cell is split into two or four, it is difficult to assign the new cell IDs while maintaining the monotonicity of the cell IDs. The SFC is a solution to split cells “locally” without affecting the global ordering of the cell IDs, but they may only work for uniform grids.

To simplify our design, we first consider a fixed grid partitioning based on some initial data, e.g., historical GPS data. Then, we discuss the update mechanism in Section 3.3 to handle the case where the grid is updated.

Our solution to the cell numbering problem is to employ a simple one-dimensional traversal curve (not necessarily a space-filling curve). Following the approach in [6], we adopt a zig-zag shaped traversal pattern (distinct from the Z-curve SFC) to index the grid cells. This method ensures that each cell is assigned a unique cell ID that increases monotonically across both dimensions, as illustrated in Fig. 3. The detailed steps of the initialization of the cell grid is as follows:

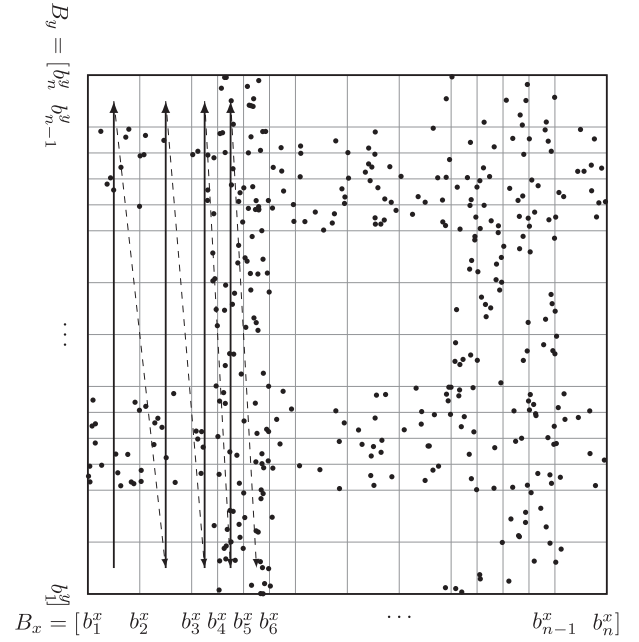


Fig. 3. Clustered data distribution.

Preparation. Prior to index construction, a full scan of the dataset is performed to collect statistics on the data distribution. This preprocessing step produces two sorted arrays: X , containing the first-dimensional coordinates, and Y , containing the second-dimensional coordinates of all data points.

Density based partitioning. Given the dataset D , the first step in building the index structure is to partition the service area into a grid. This partitioning aims to achieve a balanced distribution of data entries across grid cells. To this end, we define a threshold N_c that limits the number of data records in each grid column and row, thereby ensuring efficient last-mile searches. For example, if we wish to limit the average number of records per cell to n , the threshold can be estimated as $N_c = \sqrt{\frac{\text{total number of records}}{n}} \times n$.

The partitioning algorithm takes as input the sorted coordinate arrays X and Y (obtained during preprocessing) and produces two sorted boundary arrays: $B_x = [b_1^x, b_2^x, \dots, b_m^x]$ and $B_y = [b_1^y, b_2^y, \dots, b_n^y]$, which define the vertical and horizontal grid lines, respectively (as illustrated in Fig. 3).

The partitioning procedure is implemented as a recursive function called `Partition()`. This function operates on a reference to the sorted array X and the threshold N_c , along with the interval $[l, r]$ specifying the current segment of the array. The function recursively partitions this interval into two sub-intervals with an equal number of data points. Specifically, it computes the midpoint index m , identifies the corresponding value $X[m]$, and adds it to the boundary array B_x . The recursion continues until each sub-interval contains fewer than N_c data points. The complete algorithm for generating the vertical boundaries B_x is provided in Algorithm 1.

As shown in Fig. 3, the boundaries B_x and B_y define a virtual partitioning of the service area into a grid, where the spacing of grid lines adapts to the underlying data distribution.

3.2.2. Physical data storage

In the context of physical data storage, we use the term *data block* to refer to the serialized data form of a grid cell. Each data block corresponds to a single grid cell and physically stores all spatial data records that fall within the spatial extent of that cell. In our algorithms, we denote a cell, or equivalently the corresponding data block, as $C_{x,y}$.

Algorithm 1 GetBoundary(X, N_c)

Require: X , the sorted array of first-dimension coordinates of all data points; X_{max} , the maximum value in array X ; N_c , the threshold of the number of data records in each column of the grid.

Ensure: B_x , a sorted array that defines the vertical boundaries in the grid.

```

1:  $B_x \leftarrow [0, X_{max}]$ 
2: function PARTITION( $X, N_c, l, r$ )
3:   if  $r - l \leq N_c$  then
4:     return
5:   end if
6:    $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
7:    $B_x.insert\_sorted(X[m])$ 
8:   PARTITION( $X, N_c, l, m$ )
9:   PARTITION( $X, N_c, m, r$ )
10: end function
11: PARTITION( $X, N_c, 0, X.size$ )
12: return  $B_x$ 

```

since the two can be used interchangeably in most scenarios. x and y are the column and row IDs of the grid cell, respectively. We can also refer to a cell by its linear cell ID along the Zig-zag curve, denoted as C_i .

After virtually partitioning the service area using the boundary arrays B_x and B_y , we must physically organize the original spatial data records into the corresponding data blocks. This requires scanning the dataset again and, for each spatial record p_i , identifying the appropriate block $C_{x,y}$.

Since the grid cells are non-uniform in size, we cannot directly compute the cell ID of p_i from its coordinates. Instead, we must search the boundary arrays B_x and B_y to determine the grid cell to which p_i belongs. This reduces to two one-dimensional search problems. To accelerate this process, we construct learned indices on B_x and B_y to predict the cell ID of a given spatial point. We will discuss the design of the one-dimensional learned index in detail in the following subsection.

Assuming that the cell ID of p_i is correctly determined and denoted as C_i , DGLSI proceeds as follows. It checks both the in-memory cache and the manifest file to determine whether the corresponding block $C_{x,y}$ has already been created:

- If the block does not exist, a new block is instantiated with its spatial range derived from the boundary arrays B_x and B_y , and assigned the cell ID C_i . The data record p_i is then inserted into this block. The block remains in the in-memory cache until it either reaches its capacity or has not been accessed for a prolonged period. At that point, it is serialized and written to the corresponding file.
- If the block already exists, it is loaded into the memory cache (if not already present), and the data record p_i is inserted into it.

Algorithm 2 presents the pseudocode for this final stage of the index-building process.

3.2.3. One-dimensional learned index

The concept of a one-dimensional learned index is straightforward: using the boundary array B_x , we construct a two-stage Recursive Model Index (RMI) [1], denoted as $M_x()$, following the methodology introduced in [1]. Given a query point $p_i = (x_i, y_i)$, the goal is to locate the interval $[b_l^x, b_r^x]$ in B_x such that $x_i \in [b_l^x, b_r^x]$.

The model $M_x()$ takes x_i as input and predicts the approximate index idx of the corresponding boundary element in B_x . To determine the exact column that contains x_i , DGLSI examines $B_x[idx]$ along with its neighboring elements, such as $B_x[idx - 1]$ and $B_x[idx + 1]$, and

Algorithm 2 Store(D, B_x, B_y)

Require: D , the dataset of spatial points; B_x, B_y , the boundary arrays that define the grid; $\mathcal{G}$, the set of data groups, a.k.a. cells, that has already been created; M_c , the memory cache that stores several data groups.

```

1:  $M_c \leftarrow \{\}$ 
2: for  $p \in D$  do
3:    $C_i \leftarrow \text{FindCellID}(p, B_x, B_y)$ 
4:   if  $G_{C_i} \notin \mathcal{G}$  then
5:      $G_{C_i} \leftarrow \text{CreateGroup}(C_i)$ 
6:     INSERTINTOCACHE( $G_{C_i}, M_c$ )
7:   end if
8:   if  $G_{C_i} \notin M_c$  then
9:      $G_{C_i} \leftarrow \text{LoadGroup}(C_i)$ 
10:    INSERTINTOCACHE( $G_{C_i}, M_c$ )
11:   end if
12:    $G_{C_i}.insert(p)$ 
13: end for
14: function INSERTINTOCACHE( $G, M_c$ )
15:   if  $M_c.size > \text{cache\_size}$  then
16:      $G_{lru} \leftarrow \text{FindLRU}(M_c)$ 
17:      $G_{lru}.SERIALIZE()$ 
18:      $M_c.remove(G_{lru})$ 
19:   end if
20:    $M_c.insert(G)$ 
21: end function

```

performs a bounded search to find the correct interval that satisfies $b_l^x \leq x_i < b_r^x$.

The process for locating the row index in the y -dimension is analogous, using the boundary array B_y and a corresponding model $M_y()$.

Once both the column and row IDs are determined, the cell ID of p_i is computed using a simple formula that reflects the ZigZag indexing pattern:

$$C_i = col_id \times num_rows + row_id, \quad (1)$$

where col_id and row_id are the resolved indices for the x and y dimensions, respectively, and num_rows denotes the total number of rows in the grid. This solved the first sub-task derived from the “Goal of DGLSI” (Definition 2). For the second sub-task, we train an additional linear learned index using existing RMI method to map the linear cell ID C_i to its physical storage position.

3.3. Update mechanism

Recall the “Goal of DGLSI” defined in Definition 2, which states that DGLSI should efficiently determine the physical storage position of a grid cell given a spatial coordinate. Thus far, we have discussed how to achieve this goal under the strict assumption that the dataset is static — that is, no new data entries are inserted into the system after the initial index construction.

In this subsection, we explore how DGLSI can be extended to support dynamic updates. A natural approach is to adopt the concept of out-of-place updates for handling new data insertions. Specifically, incoming data entries are first inserted into an in-memory buffer, allowing the global learned index and the pre-established cell grid to remain unchanged initially. This approach avoids the overhead of retraining the model or re-partitioning the grid upon each insertion. When the buffer reaches a predefined threshold, the system merges the buffered data into the persistent storage. If the merged data causes the size of any grid cell to exceed a specified limit, thus violating the design expectation of a balanced number of data entries per cell (as discussed in Section 3.2.1), the grid may require re-partitioning.

In the following, we elaborate on the detailed mechanisms that enable efficient and consistent updates in DGLSI.

In-memory delta-data block. We associate a delta data block, denoted as ΔC_i , with each original data block C_i . The delta data block ΔC_i is a temporary in-memory storage unit that corresponds to the same grid cell as C_i . When a new data record p is inserted into the system, the system first locates the appropriate delta data block ΔC_i , inserts p into it. This approach allows the system to reuse the existing grid cell ID i to index the in-memory delta data block.

As new data records accumulate, the delta data block ΔC_i may eventually exceed a predefined size threshold. When this occurs, the system triggers a background process to merge the delta data block ΔC_i with its corresponding original data block C_i .

This leads to a key design challenge: merging increases the size of C_i , which can eventually degrade the performance of linear search within the block. One of the core assumptions in DGLSI is that each data block is sufficiently small to enable efficient linear search without requiring additional indexing structures.

Once this assumption is violated, the block C_i may need to be split. However, the current grid layout is both “dense” and continuous, meaning all cell identifiers are already allocated without gaps, leaving no space to insert new cells. Consequently, the system must re-partition the grid to accommodate the updated data distribution.

Re-partitioning the grid. In the previous sections, we assumed that an initial full dataset scan is available for constructing the grid. However, this assumption may not always hold in practice for two main reasons: (1) performing a full scan can be prohibitively expensive, and (2) the initial dataset may be too small to accurately represent the long-term data distribution, especially if subsequent updates significantly alter the distribution. In such cases, re-partitioning the grid becomes necessary to maintain the efficiency of the index structure.

When the spatial distribution of newly inserted data is highly skewed, certain data blocks may grow excessively large over time. To address this, we monitor the sizes of data blocks within each column and row. If a sufficient number of blocks in a particular column or row exceed a predefined size threshold, the corresponding column or row will be re-partitioned – typically by splitting it into two sub-columns or sub-rows – to rebalance the grid and restore query efficiency.

In this subsection, we present the re-partitioning algorithm designed to handle such dynamic updates in a principled and efficient manner.

To support the data insertion mechanism, we maintain two auxiliary arrays, N_x and N_y , which track the total number of data entries in each column and row of the grid, respectively. The dimensions of N_x and N_y are always aligned with those of the boundary arrays B_x and B_y . Whenever a new data record p is inserted into the system, the corresponding delta data block ΔC_i is updated, and the appropriate counters in N_x and N_y are incremented accordingly. If the number of data entries in any column or row exceeds a predefined threshold, the respective column or row is re-partitioned to maintain balance and ensure query efficiency.

Importantly, we observe that a full scan of the dataset is not strictly required to construct or re-partition the grid. Instead, we may randomly down-sample the dataset to obtain an approximate representation of the data distribution. This approximate distribution is sufficient to guide the construction or re-partitioning of the grid in a lightweight and efficient manner.

Algorithm 3 presents the re-partitioning procedure based on the down-sampled data.

In Algorithm 3, a new boundary m is added to B_x between $B_x[i]$ and $B_x[i + 1]$, such that the number of data entries in $[B_x[i], m)$ is the same as that in $[m, B_x[i + 1])$. The array N'_x is updated accordingly to reflect the new partitioning.

Algorithm 3 RePartition(N_x, B_x)

Require: N_x, B_x ; assuming that $N_x[i] > N_c$ for some i .

Ensure: new N'_x , new B'_x .

- 1: Scan a randomly down-sampled subset of the column x to get the middle m such that the number of data entries satisfying $B_x[i] \leq x < m$ is the same as that satisfying $m \leq x < B_x[i + 1]$.
 - 2: $B'_x \leftarrow B_x[0, i] \cup \{m\} \cup B_x[i + 1, B_x.size]$
 - 3: $N'_x \leftarrow N_x[0, i - 1] \cup \{N_x[i]/2, N_x[i]/2\} \cup N_x[i + 1, N_x.size]$
-

Re-allocate the data records with new partitioning scheme. After the “virtual” grid is re-partitioned, the physical storage layout must also be updated to reflect the new grid structure. Typically, re-partitioning affects only a limited number of grid cells. For each affected cell, the system loads its corresponding data block from persistent storage, reassigns each data record to a new cell based on the updated partition boundaries, and regenerates the data blocks accordingly. The newly generated blocks are then written back to storage, completing the synchronization between the virtual and physical layouts.

4. Query processing

4.1. Nearest neighbor query

Given the query $Q(p_c, r)$, the process of finding the nearest neighbor of p_c unfolds as follows:

1. Determine the cell ID of p_c using the one-dimensional learned indices.
2. Load the data block $C_{x,y}$ containing p_c into memory. If a delta data block $\Delta C_{x,y}$ exists, it is merged with the original block. For simplicity, we denote the merged block as $C_{x,y} \leftarrow C_{x,y} \cup \Delta C_{x,y}$.
3. Perform a linear search over $C'_{x,y}$ to identify the nearest neighbor of p_c within the cell.
4. The nearest neighbor of p_c may not reside within the same cell. In such cases:
 - (a) Suppose a candidate point $p_i \in C_{x,y}$ is found such that $d(p_i, p_c) \leq r$. A circle centered at p_c with radius $d(p_i, p_c)$ is drawn. If this circle intersects the boundary of cell $C_{x,y}$, neighboring cells must be searched for potentially closer points.
 - (b) If p_c is the only point in the cell, all eight adjacent cells are loaded. A search is conducted in these neighboring cells, and if at least one candidate is found, the procedure reverts to case (a).

Algorithms 4 and 5 present the pseudocode for processing the nearest neighbor query. Algorithm 4 ultimately identifies the nearest neighbor of p_c within the so-called “XCell” (signifying the extended group encompassing an original group and its eight surrounding groups). Subsequently, Algorithm 5 extends the search area further if the distance between p_c and the nearest neighbor discovered exceeds the distance between p_c and the boundaries of the XCell. Here, $XC.s$, $XC.n$, $XC.w$, and $XC.e$ denote the south, north, west, and east boundaries of the XCell, respectively.

4.2. Range query

Given the query $Q(p_1, p_2)$, the process of identifying all spatial points contained within the rectangle defined by p_1 and p_2 involves the following steps:

1. The cell IDs for p_1 and p_2 are determined using one-dimensional learned indices.

Algorithm 4 NearestNeighbor(p_c, r), Part I**Require:** p_c , the query point; r , the radius of the query.

```

1:  $C \leftarrow \text{FindCellID}(p_c, B_x, B_y)$ 
2:  $C_{x,y} \leftarrow \text{LoadGroup}(C)$ 
3:  $XC \leftarrow C_{x,y}$ 
4:  $d = r$ 
5:  $p \leftarrow \text{LinearSearch}(C'_{x,y}, p_c, r)$ 
6:  $d = \min(d, d(p, p_c))$ 
7: while  $p = \text{NULL}$  do
8:    $N \leftarrow \text{FindNeighbors}(XC)$ 
9:   for  $C' \in N$  do
10:     $C_{x',y'} \leftarrow \text{LoadGroup}(C')$ 
11:     $p' \leftarrow \text{LinearSearch}(C_{x',y'}, p_c, r)$ 
12:    if  $p' \neq \text{NULL}$  then
13:       $d' = d(p', p_c)$ 
14:      if  $d' < d$  then
15:         $d = d'$ 
16:         $p = p'$ 
17:      end if
18:    end if
19:  end for
20:   $XC \leftarrow XC \cup \{\text{neighbors}\}$ 
21: end while

```

Algorithm 5 NearestNeighbor(p_c, r), Part II**Require:** p_c , the query point; r , the radius of the query; p , the “currently” found nearest neighbor of p_c ; XC , the XCell that contains p_c and p . d , the distance between p and p_c .

```

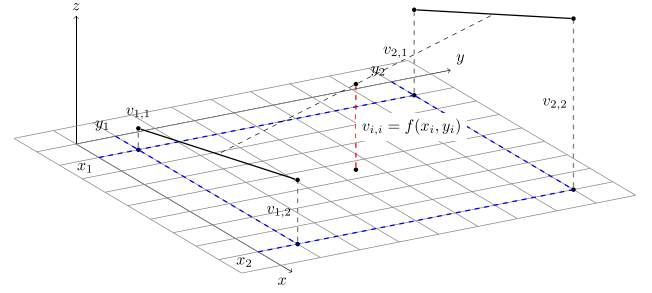
1: while  $d(p_c, p) > \text{any of } d(p_c, XC.s), d(p_c, XC.n), d(p_c, XC.w), d(p_c, XC.e)$  do
2:    $N \leftarrow \text{FindNeighbors}(XC)$ 
3:   for  $C' \in N$  do
4:     $C_{x',y'} \leftarrow \text{LoadGroup}(C')$ 
5:     $p' \leftarrow \text{LinearSearch}(C_{x',y'}, p_c, r)$ 
6:    if  $p' \neq \text{NULL}$  then
7:       $d' = d(p', p_c)$ 
8:      if  $d' < d$  then
9:         $d = d'$ 
10:        $p = p'$ 
11:      end if
12:    end if
13:  end for
14:   $XC \leftarrow XC \cup \{\text{neighbors}\}$ 
15: end while

```

2. The range of cell IDs that intersect the rectangle defined by p_1 and p_2 is computed.
3. The corresponding data groups are loaded into the memory cache, and a linear search is performed within each group.
4. (1) For groups that partially overlap with the query range, each data record within the group is inspected to confirm its inclusion within the query range. (2) For groups entirely encompassed by the query range, all data records are added directly to the result set.

5. Evaluation**5.1. Dataset and experimental setup**

We utilize the Geolife dataset version 1.3, collected and published by researchers at Microsoft Research. This dataset comprises GPS trajectories from 182 users collected over a period of more than five years (2007–2012). The majority of the spatial points are concentrated in

**Fig. 4.** Bilinear interpolation function.

the central urban area of Beijing, China. The dataset includes 17,621 trajectories, containing a total of 24,876,978 spatial points, which exhibit a non-uniform distribution due to clustering along the road network. During the index construction phase, we set the expected average number of data records per cell to $n = 10$.

The experiments were conducted on a desktop computer equipped with an Intel Core i7-9700K CPU, 64 GB of RAM, and a Samsung 980 Pro 1 TB NVMe SSD. We evaluated the storage overhead and query performance of the proposed index structure, comparing it against SPRIG [6], a similar grid-based index. The DGLSI index was implemented in C++, alongside a simplified version of SPRIG, which shares the same `record_t` structure and manifest file. The manifest file stores the mapping table that links cell IDs to the byte offsets of their corresponding groups in the data file. The primary differences between DGLSI and SPRIG lie in the choice of learned index structure (per-dimensional learned models versus spatial interpolation functions) and the data partitioning strategy. Before presenting the evaluation results, we provide a brief overview of SPRIG.

5.2. Baseline model: Simplified SPRIG

SPRIG employs a spatial interpolation function as the learned model to predict the cell ID of a spatial point. Spatial interpolation is a widely used method in geographic information systems (GIS) and image processing, estimating the value of a variable at any location within a service area based on measured values at observed locations [10,11]. Given a set of 2-D observed points $p_1, p_2, \dots, p_n$, where $p_i = (x_i, y_i)$, and their corresponding values $v_1, v_2, \dots, v_n$, the spatial interpolation function $f(x, y)$ estimates the value at a location (x, y) based on the observed values.

Fig. 4 illustrates the bilinear interpolation function $f(x, y)$ used in SPRIG:

$$f(x, y) = \begin{bmatrix} 1-x & x \end{bmatrix} \begin{bmatrix} f(x_1, y_1) & f(x_1, y_2) \\ f(x_2, y_1) & f(x_2, y_2) \end{bmatrix} \begin{bmatrix} 1-y \\ y \end{bmatrix} \quad (2)$$

$$= \phi_0 + \phi_1 x + \phi_2 y + \phi_3 xy,$$

where $\phi_0, \phi_1, \phi_2, \phi_3$ are coefficients trained on the observed data points. These coefficients can be calculated by solving a linear system of equations based on the values at the four corners $f(x_1, y_1), \dots, f(x_2, y_2)$. Using the established grid and the “Zig-Zag” cell ID numbering scheme illustrated in Fig. 3 and Section 3.2.3, the bilinear interpolation function can be estimated by minimizing the sum of squared errors (SSE) as the loss function.

The query process using the bilinear interpolation function is straightforward. Given a query point $p_i = (x_i, y_i)$, the cell ID of p_i is estimated using the interpolation function $C_{pred} = f(x_i, y_i)$. Let $C_{pred,x}$ and $C_{pred,y}$ represent the predicted column and row IDs, respectively. Two error bounds, ϵ_x and ϵ_y , for the x and y dimensions are derived during the training process, typically based on the maximum prediction error of the function. The real cell ID of p_i is determined by performing a model-biased search within the range $[C_{pred,x} - \epsilon_x, C_{pred,x} + \epsilon_x] \times [C_{pred,y} - \epsilon_y, C_{pred,y} + \epsilon_y]$.

Table 2
Query performance of DGLSI and SPRIG (milliseconds).

(a) Nearest neighbor query				(b) Range query			
R_p	T_d	T_s	T'_d	R_r	T_d	T_s	T'_d
200	3.77	3.89	3.92	200	15.43	17.31	30.58
400	8.16	8.52	9.17	400	50.26	58.19	100.34
800	13.24	14.63	15.21	800	173.98	183.73	279.32
1600	19.65	21.80	20.08	1600	549.42	568.64	728.81

5.3. Storage overhead

The storage overhead is calculated by comparing the size of the indexed dataset to that of the raw dataset, which consists solely of spatial points. The raw dataset size is 1.67 GB, while the indexed dataset using DGLSI occupies 1.93 GB, resulting in a storage overhead of less than 15.6%. The majority of this overhead arises from metadata required to locate data groups in the storage system, such as the manifest file. Additionally, the learned Recursive Model Index (RMI) models for each dimension contribute to the overhead. In contrast, SPRIG incurs negligible overhead for its learned model, as the bilinear interpolation function can be represented using only four floating-point numbers.

5.4. Query performance

5.4.1. Query workload and data distribution

This study assumes that the query workload is known in advance, enabling optimization of the index structure. The following parameters characterize the query workloads:

1. *Nearest neighbor query workload*: The average radius R_p (in meters) of all nearest neighbor queries.
2. *Range query workload*: The average query area R_r (in square meters) of all range queries, calculated as the geometric mean of the width and height of the query rectangle.

Queries are randomly generated based on the parameters R_p and R_r . For nearest neighbor queries, query points are randomly distributed within the service area, and the radius of the query circle follows a Gaussian distribution with mean R_p and standard deviation $0.1 \times R_p$. For range queries, the width and height of the query rectangle follow a Gaussian distribution with mean R_r and standard deviation $0.5 \times R_r$.

5.4.2. Results

Table 2 summarizes the query performance of DGLSI and SPRIG on the Geolife dataset under varying query workloads. Query performance is measured as the average execution time of 100,000 queries, expressed in milliseconds. The execution times for read-only DGLSI, SPRIG, and extended DGLSI are denoted as T_d , T_s , and T'_d , respectively.

We observed slightly improved query performance of DGLSI compared to SPRIG in both nearest neighbor and range queries. One notable limitation of SPRIG is the relatively large per-dimension error bounds ϵ_x and ϵ_y . These arise because the row IDs of the cells, as illustrated in Fig. 3 and Eq. (1), may vary significantly along the y -axis while the encoded cell ID remains monotonically increasing. This phenomenon reduces the effectiveness of the error bounds in pruning the search space. In contrast, DGLSI employs per-dimension learned indices to predict cell IDs, enabling better adaptation to complex data distributions.

However, the extended DGLSI demonstrates slightly worse query performance compared to the read-only DGLSI. This is primarily due to the extended DGLSI involving additional data groups during query processing, which results in more complex and resource-intensive search operations.

6. Related work

6.1. Learned index

Traditional indexes operate without assumptions about data distributions. However, leveraging distribution information can significantly reduce storage overhead without a substantial increase in search time. Consider a simple key–value store with continuous integer keys from 1 to N and fixed-size data entries. Traditional index structures, such as B-Trees, might require $\mathcal{O}(N)$ additional nodes to index the entire dataset. In contrast, a learned index can “predict” the position of any record with $\mathcal{O}(1)$ space overhead. For instance, a simple linear function, $pos = M(k) = wk + b$, where k is the lookup key and w, b are trainable model parameters, can serve as the learned index model. Existing research [1] has demonstrated the efficiency of learned indexes in reducing storage overhead for even more complex data distributions, often without compromising query performance when compared to traditional index structures like B-Trees and hash indexes.

In this paper, we define an index as a mechanism that exhibits no false negatives. This means that if a data record exists in storage, the index must precisely locate and retrieve it upon a query. While learned models inherently can make errors, these inaccuracies are typically manageable within the context of sorted datasets. Most learned index architectures mitigate these errors by incorporating a local search (also known as a “last-mile search”) after the initial prediction, thereby ensuring the correctness of the retrieved result.

6.2. Spatial point indexes

One of the widely used spatial indexes in popular spatial databases, the R-tree, is predicated on the concept of bounding boxes for spatial objects and operates via the branch and bound (BB) principle [12]. To construct an R-tree index, the service area is hierarchically subdivided into smaller regions, typically rectangles, which are then organized into a tree structure.

On the other hand, SFC index spatial data by recursively dividing the service area into a hierarchy of cell grids, each split resulting in four equal parts. Each cell within this hierarchy is assigned an ID according to the specific SFC used, such as the Z-curve or Hilbert curve. For instance, at the first hierarchical level, four cells covering the entire service area might have binary IDs 00, 01, 10, 11, with their order determined by the SFC’s mapping (e.g., clockwise, counterclockwise, or ZigZag). Extending this, splitting a cell into four smaller ones for the next level simply appends two bits to its existing ID (e.g., cell 00 subdivides into 0000, 0001, 0010, 0011). This inherent property of SFCs allows for achieving very high resolution, theoretically yielding minuscule leaf-level cells containing only a handful of data entries. A crucial characteristic of SFC-based indexes is the regularity of their cell grid; at any given level, all cells are rectangular with predefined edge dimensions. This regularity enables the calculation of a cell ID for any given location p at a specific level l in $\mathcal{O}(1)$ time.

A notable limitation of SFC-based spatial indexes is their inability to adaptively partition cells based on data distribution. This results in significant variability in the number of data entries per cell, complicating the determination of an optimal grid level. If leaf-level cells are excessively large, query performance may degrade due to the high

volume of data entries requiring individual examination. Conversely, overly small cells can lead to a proliferation of examined cells (often with discontinuous cell IDs) during a spatial range query.

In summary, R-tree-based indexes offer flexibility in adapting node spatial ranges to data distribution, while SFC-based indexes excel at efficiently locating exact spatial points owing to their inherent regular cell grid structure.

Recent works [5,6,13–15] have investigated the use of learned models for spatial indexing to leverage underlying data distributions. Among these, [5] proposed the ZM-index, which maps spatial data into a linear order using a space-filling curve (SFC) and then applies a learned linear index on the resulting one-dimensional data. However, this approach inherits the limitations of SFCs, as previously discussed. [6,13] introduced a grid-based method that employs a spatial interpolation function, learned directly to facilitate spatial point indexing. Unlike using two separate one-dimensional learned indexes, this approach utilizes a single uniform learned index. SLBRIN [14] is another grid-based index that organizes data using an SFC-based grid, specifically Geohash. It introduces the concepts of history range (HR) and current range (CR) to support efficient query processing and data insertion, respectively. These concepts resemble out-of-place updates and provide valuable insights for our future work, despite employing different data partitioning strategies. Even with different data partitioning strategies.

7. Conclusions and future work

This paper introduces DGLSI, a learned spatial index structure specifically designed for clustered spatial data. This index is built upon an adaptive grid-based partitioning strategy that considers the underlying data distribution. We evaluated the query performance of DGLSI using the Geolife dataset and demonstrated its superior performance over SPRIG, a comparable grid-based index, in both nearest neighbor and range queries. Furthermore, DGLSI exhibits a storage overhead of less than 15.6% relative to the raw dataset. We also propose an update mechanism for DGLSI that leverages the Delta Data Group and a repartitioning strategy, drawing inspiration from LSM-trees.

While learned index structures offer significant potential in predicting data record storage positions, we acknowledge that the performance of DGLSI and similar spatial index structures is highly dependent on the employed data partitioning, file organization, and caching strategies. The current re-partitioning strategy preserves the original grid structure; specifically, when a column is split by introducing a new boundary, all grid cells within that column are uniformly divided, which is often unnecessary and inefficient. In future work, we aim to develop more advanced learned index structures that support dynamic and adaptive data partitioning. For example, we plan to explore partitioning strategies based on the temporal attributes of data entries. Additionally, we plan to investigate the impact of integrating file caching strategies with the learned model.

Furthermore, while data update support has been incorporated, the current design of DGLSI does not account for the temporal semantics inherent in spatial data. For instance, in GPS trajectory datasets, the chronological order of data points is crucial for understanding the movement patterns of monitored objects. However, DGLSI currently treats all data points as independent spatial records. A promising future direction involves indexing both spatial and temporal dimensions, which would enable more sophisticated pattern queries for trajectory data.

CRedit authorship contribution statement

Xiaofei Zhao: Writing – original draft, Software, Methodology, Conceptualization. **Kam-Yiu Lam:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The test dataset is publicly available.

References

- [1] T. Kraska, A. Beutel, E.H. Chi, J. Dean, N. Polyzotis, The case for learned index structures, in: Proceedings of the 2018 International Conference on Management of Data, in: SIGMOD '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 489–504, <http://dx.doi.org/10.1145/3183713.3196909>.
- [2] A. Guttman, R-trees: A dynamic index structure for spatial searching, in: Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data, in: SIGMOD '84, Association for Computing Machinery, New York, NY, USA, 1984, pp. 47–57, <http://dx.doi.org/10.1145/602259.602266>.
- [3] N. Beckmann, H.-P. Kriegel, R. Schneider, B. Seeger, The R*-tree: An efficient and robust access method for points and rectangles, in: Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, in: SIGMOD '90, Association for Computing Machinery, New York, NY, USA, 1990, pp. 322–331, <http://dx.doi.org/10.1145/93597.98741>.
- [4] T. Asano, D. Ranjan, T. Roos, E. Welzl, P. Widmayer, Space-filling curves and their use in the design of geometric data structures, Theoret. Comput. Sci. 181 (1) (1997) 3–15, [http://dx.doi.org/10.1016/S0304-3975\(96\)00259-9](http://dx.doi.org/10.1016/S0304-3975(96)00259-9).
- [5] H. Wang, X. Fu, J. Xu, H. Lu, Learned index for spatial queries, in: 2019 20th IEEE International Conference on Mobile Data Management, MDM, 2019, pp. 569–574, <http://dx.doi.org/10.1109/MDM.2019.00121>.
- [6] S. Zhang, S. Ray, R. Lu, Y. Zheng, SPRIG: A learned spatial index for range and kNN queries, in: 17th International Symposium on Spatial and Temporal Databases, in: SSTD '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 96–105, <http://dx.doi.org/10.1145/3469830.3470892>.
- [7] B. Zou, M. Zeng, C. Zhu, L. Xiao, Z. Chen, A learned prefix bloom filter for spatial data, in: C. Strauss, A. Cuzzocrea, G. Kotsis, A.M. Tjoa, I. Khalil (Eds.), Database and Expert Systems Applications, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2022, pp. 336–350, http://dx.doi.org/10.1007/978-3-031-12423-5_26.
- [8] H.K. Dai, H.C. Su, On the locality properties of space-filling curves, in: T. Ibaraki, N. Katoh, H. Ono (Eds.), Algorithms and Computation, Springer, Berlin, Heidelberg, 2003, pp. 385–394, http://dx.doi.org/10.1007/978-3-540-24587-2_40.
- [9] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil, The log-structured merge-tree (LSM-tree), Acta Inform. 33 (4) (1996) 351–385, <http://dx.doi.org/10.1007/s002360050048>.
- [10] Q. Tan, X. Xu, Comparative analysis of spatial interpolation methods: An experimental study, Sensors Transducers 165 (2) (2014) 155.
- [11] C. Boberg, J. Nerbonne, D. Watt, The Handbook of Dialectology, John Wiley & Sons, 2018.
- [12] A.H. Land, A.G. Doig, An automatic method of solving discrete programming problems, Econometrica 28 (3) (1960) 497–520, <http://dx.doi.org/10.2307/1910129>, [arXiv:1910129](https://arxiv.org/abs/1910129).
- [13] S. Zhang, S. Ray, R. Lu, Y. Zheng, Efficient learned spatial index with interpolation function based learned model, IEEE Trans. Big Data 9 (2) (2023) 733–745, <http://dx.doi.org/10.1109/TBDATA.2022.3186857>.
- [14] L. Wang, L. Hu, C. Fu, Y. Yu, P. Tang, F. Zhang, R. Liu, SLBRIN: A Spatial Learned Index Based on BRIN, ISPRS Int. J. Geo-Inf. 12 (4) (2023) 171, <http://dx.doi.org/10.3390/ijgi12040171>.
- [15] J. Qi, G. Liu, C.S. Jensen, L. Kulik, Effectively learning spatial indices, Proc. VLDB Endow. 13 (12) (2020) 2341–2354, <http://dx.doi.org/10.14778/3407790.3407829>.